

Enterprise Knowledge Intelligence Platform

Team Members

- M. Sathya Krishna – 2510080006.
- Yashwanth – 2510080001.
- Aneeq – 2510080005.
- Abhinav – 2510080002.

Guide: Anitha. P

An advanced algorithm-driven system for intelligent document processing, semantic search, and knowledge discovery — built on rigorous data structures and algorithmic foundations.

The Problem with Traditional Search

Inefficient Search

Keyword matching misses semantic intent and contextual relevance

No Fuzzy Handling

Spelling errors and variant queries return zero results

Slow at Scale

Linear scans across large document collections are prohibitively slow

No Optimisation

Task assignment and scheduling lack algorithmic intelligence



Project Objectives

TextHack unifies algorithmic rigour with enterprise-grade engineering to deliver a platform that is fast, intelligent, and scalable.

1

Intelligent Query Engine

Build the TextHack Algorithm Engine for fast, multi-pattern document search

2

Fuzzy & Semantic Search

Enable typo-tolerant queries using Dynamic Programming and vector embeddings

3

Document Similarity

Apply suffix structures and LCP arrays for precise similarity analysis

4

Graph-Based Assignment

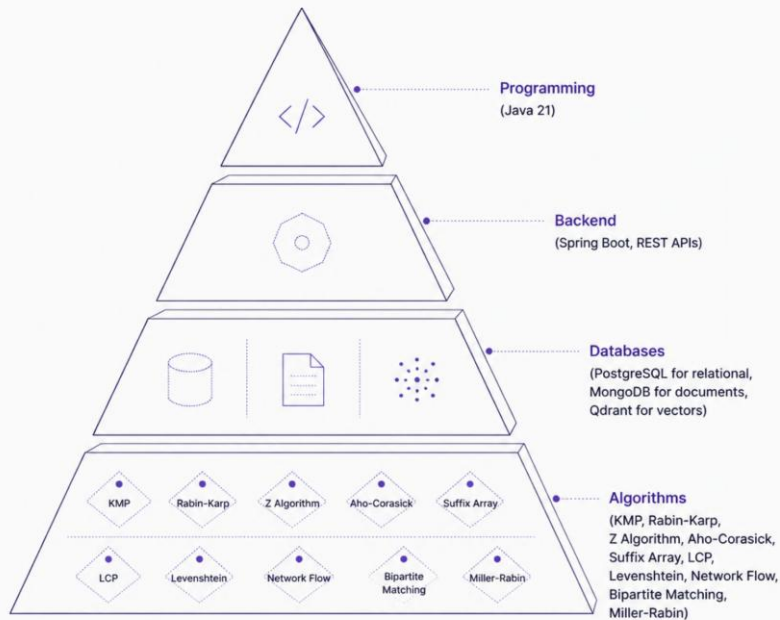
Use network flow and bipartite matching for reviewer and task allocation

5

Optimisation & Randomisation

Demonstrate approximation and randomized algorithms for scheduling and indexing

Technology Stack



Core Technologies

- **Java 21** — primary language for all algorithm implementations
- **Spring Boot** — RESTful backend with role-based authentication
- **PostgreSQL** — structured relational data storage
- **MongoDB** — flexible document-based storage
- **Qdrant** — high-performance vector database for semantic search
- **Git & Maven** — version control and build automation

Expected Outcomes

TextHack will deliver a production-ready knowledge platform combining algorithmic precision with modern software architecture.



Fast Pattern Search

Multi-keyword matching with KMP, Rabin-Karp, and Aho-Corasick



Semantic & Fuzzy Search

AI-assisted vector search with typo correction via Levenshtein Distance



Document Similarity

Suffix Array and LCP-based detection of near-duplicate content



Graph-Based Assignment

Reviewer and task allocation using Ford-Fulkerson and bipartite matching



Secure REST APIs

Role-based authentication with scalable hybrid database architecture

Project Modules

Six modular components, each encapsulating a distinct algorithmic domain.

Module 1 — Query Engine

Pattern search, query processing, document indexing

Module 2 — String Algorithms

KMP, Rabin-Karp, Z Algorithm, Aho-Corasick, Suffix & LCP Arrays

Module 3 — Dynamic Programming

Levenshtein, Damerau-Levenshtein, sequence alignment, similarity

Module 4 — Graph Algorithms

Ford-Fulkerson, Edmonds-Karp, Dinic's, bipartite matching, citation flow

Module 5 — Optimisation

Approximation algorithms, task scheduling, reviewer assignment

Module 6 — Randomised

Miller-Rabin primality test, universal hashing, reservoir sampling

Project Timeline

An eight-week sprint plan moving from system design through algorithm implementation to full deployment.

